

Sistema Híbrido de Recuperación de Información y Generación Aumentada para Noticias Tecnológicas

Alberto E Marichal Fonseca
Amalia Gonzales Ortega
Katherine Rodríguez Rodríguez

Universidad de La Habana
`alberto.emarichal1@estudiantes.matcom.uh.cu`
`amalia.gonzales1@estudiantes.matcom.uh.cu`
`katherine.rodriguez@estudiantes.matcom.uh.cu`

Resumen Este informe documenta el diseño, la implementación y la evaluación crítica de un sistema de recuperación de información con generación aumentada por recuperación (RAG) para noticias tecnológicas en español. El sistema construye su propio corpus mediante arañas Scrapy, normaliza y fragmenta los documentos, mantiene un índice invertido y una base vectorial Chroma, y combina recuperación probabilística y semántica mediante Weighted Reciprocal Rank Fusion. La respuesta final se genera con un modelo Llama 3.1 local tras una fase de retroalimentación Rocchio desde la interfaz web. Además de describir la arquitectura, el documento formaliza las ecuaciones empleadas, resume las estadísticas del corpus y expone limitaciones técnicas y mejoras futuras.

Keywords: Recuperación de Información · RAG · Modelo de Lenguaje · Chroma · Rocchio · Reciprocal Rank Fusion

1. Introducción

El objetivo del proyecto es construir un Sistema de Recuperación de Información (SRI) especializado en el dominio de tecnología de consumo, principalmente móviles, ordenadores, componentes y software. A diferencia de un buscador genérico, la solución controla el ciclo completo de datos: extracción del corpus, normalización, indexación, recuperación híbrida, retroalimentación de relevancia y generación de respuestas en español.

La motivación principal es reducir la distancia entre una consulta natural del usuario y la evidencia documental disponible. Para ello, el sistema combina técnicas clásicas de recuperación, robustas para términos exactos y referencias de producto, con recuperación densa basada en embeddings, más adecuada para similitud semántica. Finalmente, un modelo generativo local usa los documentos recuperados como contexto y evita responder fuera de la evidencia entregada.

2. Corpus y Preparación de Datos

2.1. Fuentes documentales

El corpus se recolecta desde Xataka mediante dos spiders Scrapy: `xataka_mobile` y `xataka_pc`. La primera recorre etiquetas relacionadas con smartphones, fabricantes y sistemas operativos móviles; la segunda recorre categorías de ordenadores, componentes, periféricos, monitores, portátiles, hardware y sistemas operativos de escritorio. Cada artículo se almacena como JSONL con título, URL, contenido, autor, fecha, etiquetas, fuente y metadatos.

La extracción incorpora medidas de control: respeto de `robots.txt`, profundidad máxima de rastreo, `autothrottle`, caché HTTP, límite temporal por spider y eliminación de duplicados por URL. El resultado no es únicamente un conjunto de páginas descargadas, sino una colección persistente y reproducible bajo `data/mobile` y `data/pc`.

2.2. Estadísticas del corpus

La Tabla 1 resume el estado del corpus leído desde los JSONL y del índice persistido. Las líneas JSONL incluyen capturas de varias ejecuciones; el sistema, sin embargo, deduplica por URL al construir el almacén documental.

Cuadro 1. Resumen cuantitativo del corpus y del índice.

Métrica	Valor
Líneas JSONL brutas	4508
Documentos únicos por URL	4409
Documentos únicos de móvil	2718
Documentos únicos de PC	1779
Documentos sin contenido textual	10
Longitud media por artículo bruto	290 palabras
Mediana de longitud por artículo bruto	228 palabras
Vocabulario	69966
Vocabulario normalizado	40096
Longitud media por artículo normalizado	255
Mediana de longitud por artículo normalizado	209

Los términos más frecuentes del índice son formas normalizadas y lematizadas por *stemming*; por ello aparecen raíces como `pantall`, `movil`, `cam`, `nuev`, `mejor`, `gb` y `pro`. Esta distribución confirma que el vocabulario dominante pertenece al dominio esperado: pantallas, cámaras, memoria, modelos comerciales, precios y comparativas de dispositivos.

3. Arquitectura General

La arquitectura sigue una tubería modular. Primero, Scrapy recolecta documentos y los persiste en JSONL. Después, `DocumentStore` carga las categorías y deduplica por URL. A continuación, `TextNormalizer` transforma el texto a tokens normalizados en español y `DocumentChunker` divide los artículos en ventanas solapadas. Esos chunks alimentan dos estructuras: un índice invertido para el recuperador probabilístico y una colección Chroma para recuperación vectorial.

En tiempo de consulta, `QueryProcessor` limpia y normaliza la pregunta del usuario. Si la expansión está activa, se agregan términos por coocurrencia. La consulta expandida se envía al recuperador probabilístico y al recuperador vectorial de LangChain. Sus listas se fusionan con Weighted Reciprocal Rank Fusion (WRRF). Si la relevancia local no alcanza un umbral, el sistema realiza una búsqueda externa, fragmenta esos documentos y puede incorporarlos a Chroma. La interfaz React permite marcar documentos relevantes y no relevantes; con esa señal, Rocchio reformula la consulta y se ejecuta una segunda recuperación antes de generar la respuesta final.

4. Normalización e Indexación

4.1. Normalización textual

El normalizador aplica minúsculas, eliminación de URL, limpieza de restos HTML y símbolos, eliminación de emojis, tokenización con NLTK, supresión de palabras vacías en español y *stemming* Snowball. Dado un texto bruto x , el resultado se modela como una secuencia:

$$T(x) = \langle t_1, t_2, \dots, t_n \rangle, \quad (1)$$

donde cada t_i es un término normalizado. La consulta se procesa con la misma función para mantener consistencia entre los términos del usuario y los términos indexados.

4.2. Fragmentación documental

La estrategia activa es `sliding`, con tamaño de ventana $L = 500$, solapamiento $O = 100$ y tamaño mínimo de chunk de 100 tokens. Sea $D = \langle w_1, \dots, w_n \rangle$ la secuencia de tokens de un documento. El chunk C_j se define como:

$$C_j = \langle w_{s_j}, \dots, w_{\min(s_j+L-1, n)} \rangle, \quad s_j = 1 + j(L - O). \quad (2)$$

El solapamiento reduce pérdidas de contexto en los límites de ventana y favorece que una misma evidencia pueda recuperarse aunque la consulta caiga cerca de una frontera de fragmento.

4.3. Índice invertido

El índice invertido mantiene listas de postings por término. Para cada término t y documento d , se almacena la frecuencia cruda:

$$I(t) = \{(d, f_{t,d}) \mid f_{t,d} > 0\}, \quad (3)$$

donde $f_{t,d}$ es el número de apariciones de t en d . Además se guardan metadatos por chunk: longitud, título, URL, fuente, fecha, autor y previsualización normalizada. Esta estructura permite calcular frecuencias de documento, frecuencias de colección y longitudes necesarias para el modelo de lenguaje.

5. Modelos de Recuperación

5.1. Modelo probabilístico de lenguaje

El recuperador **LMRetriever** implementa *query likelihood* con suavizado de Dirichlet [2]. Primero se estima la probabilidad de colección del término:

$$P(t \mid C) = \frac{\text{cf}(t)}{|C|}, \quad (4)$$

donde $\text{cf}(t)$ es la frecuencia total de t en la colección y $|C|$ es el número total de tokens indexados. Para un documento d , la probabilidad suavizada es:

$$P_\mu(t \mid d) = \frac{f_{t,d} + \mu P(t \mid C)}{|d| + \mu}, \quad \mu = 500. \quad (5)$$

La consulta se representa como una distribución de pesos $P(t \mid q)$. El sistema calcula una media geométrica ponderada, implementada como exponencial de log-probabilidades:

$$s_{\text{LM}}(d, q) = \exp \left(\sum_{t \in q} P(t \mid q) \log P_\mu(t \mid d) \right). \quad (6)$$

Este modelo favorece documentos que podrían haber generado la consulta y evita puntuaciones nulas para términos ausentes gracias al suavizado.

Una debilidad observada durante el uso del sistema es la tendencia a escribir consultas demasiado verbosas. Aunque el suavizado de Dirichlet evita que un documento reciba probabilidad nula cuando no contiene todos los términos, el modelo de *query likelihood* sigue acumulando una contribución logarítmica por cada término de la consulta. En consecuencia, una pregunta larga con frases de relleno, matices conversacionales o demasiadas condiciones secundarias puede diluir el peso de los términos realmente discriminantes. Este comportamiento coincide con la discusión de Zhai y Lafferty sobre la sensibilidad de los modelos de lenguaje suavizados al balance entre evidencia documental y probabilidad de colección [2]: el parámetro μ ayuda a estabilizar la estimación, pero no sustituye una formulación clara de la necesidad de información. Por ello, el prototipo incorpora en la interfaz una advertencia cuando la consulta supera un umbral de palabras y una acción simple para reducir ruido léxico antes de recuperar.

5.2. Recuperación vectorial

La recuperación semántica se apoya en Chroma y en embeddings de HuggingFace. La configuración intenta cargar el modelo MiniLM-L6-v2 de Sentence Transformers; si no está disponible, cae a un vectorizador TF-IDF persistido. Para una consulta q y un documento d , el modelo denso produce vectores:

$$\mathbf{e}_q = f_\theta(q), \quad \mathbf{e}_d = f_\theta(d). \quad (7)$$

La similitud base se expresa mediante coseno:

$$\cos(q, d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \quad (8)$$

Para documentos procedentes de búsqueda web, el sistema transforma el coseno al intervalo $[0, 1]$:

$$s_{\text{web}}(q, d) = \frac{\cos(q, d) + 1}{2}. \quad (9)$$

La parte vectorial aporta tolerancia ante sinónimos, paráfrasis y consultas redactadas de forma más conversacional.

5.3. Expansión por coocurrencia

Antes de recuperar, el sistema puede expandir la consulta usando una matriz dispersa de coocurrencias. Sea M_{ij} el número de veces que los términos i y j aparecen dentro de una ventana de tamaño h . Para cada candidato c que no pertenece a la consulta original:

$$\text{score}_{\text{cooc}}(c) = \sum_{t \in q} P(t \mid q) M_{t,c}. \quad (10)$$

Se seleccionan los m candidatos con mayor puntuación, con $m = 10$ por defecto, y se normalizan:

$$P_E(c) = \frac{\text{score}_{\text{cooc}}(c)}{\sum_{c' \in E} \text{score}_{\text{cooc}}(c')}. \quad (11)$$

La distribución final interpola la consulta original con la expansión:

$$P'(t) = \alpha P(t \mid q) + (1 - \alpha) P_E(t), \quad \alpha = 0,65. \quad (12)$$

En la implementación, los pesos se convierten de nuevo a texto repitiendo términos proporcionalmente, lo que permite reutilizar el mismo contrato de entrada para el recuperador vectorial y el recuperador LangChain.

5.4. Fusión híbrida WRRF

Las puntuaciones del modelo de lenguaje y del recuperador vectorial no son directamente comparables: una procede de probabilidades suavizadas y otra de similitud vectorial. Por ello se fusionan rankings, no valores absolutos. El sistema usa Weighted Reciprocal Rank Fusion, inspirado en RRF [3]:

$$S(d) = \sum_{r \in R} \lambda_r \frac{1}{k + \text{rank}_r(d)}, \quad k = 60. \quad (13)$$

En la configuración actual hay dos recuperadores, $R = \{\text{LM}, \text{Vec}\}$, con pesos $\lambda_{\text{LM}} = 0,5$ y $\lambda_{\text{Vec}} = 0,5$. Si un documento aparece en ambos rankings, acumula contribuciones; si aparece en uno solo, conserva una puntuación parcial. Esta estrategia es estable porque depende de posiciones relativas y no de escalas heterogéneas.

5.5. Retroalimentación Rocchio

La interfaz introduce una segunda fase: el usuario selecciona documentos relevantes y no relevantes entre los resultados iniciales. Rocchio reformula la consulta como combinación del vector original, el centroide positivo y el centroide negativo [4]:

$$\mathbf{q}_{\text{new}} = \alpha \mathbf{q}_0 + \frac{\beta}{|D_R|} \sum_{\mathbf{d} \in D_R} \mathbf{d} - \frac{\gamma}{|D_N|} \sum_{\mathbf{d} \in D_N} \mathbf{d}. \quad (14)$$

Los parámetros actuales son $\alpha = 1,0$, $\beta = 0,75$ y $\gamma = 0,15$. Tras calcular el vector, los pesos negativos o inferiores al umbral se eliminan y el vector resultante se normaliza:

$$w'_t = \frac{\max(w_t, 0)}{\sum_u \max(w_u, 0)}. \quad (15)$$

La respuesta final se genera a partir de una segunda recuperación usando los términos con mayor peso en $\mathbf{q}_{\text{new}}$.

6. Generación Aumentada por Recuperación

El generador usa `llama-cpp-python` para cargar un modelo Llama 3.1 en formato GGUF. El prompt fija tres reglas: responder solo con información del contexto, no inventar datos y contestar siempre en español. Esta restricción convierte la recuperación en la fuente de verdad del sistema.

Sea K el conjunto de documentos recuperados tras Rocchio. El contexto entregado al modelo es:

$$\mathcal{C}(q) = \text{concat}(d_1, d_2, \dots, d_k), \quad d_i \in K. \quad (16)$$

Antes de generar, el sistema estima el número de tokens del contexto, del prompt y de la pregunta. Si supera la ventana del modelo, recorta el contexto:

$$|\mathcal{C}|_{\text{tok}} \leq n_{\text{ctx}} - |p|_{\text{tok}} - |q|_{\text{tok}}, \quad n_{\text{ctx}} = 2048. \quad (17)$$

El flujo evita enviar más evidencia de la que cabe en la ventana y reduce fallos por prompts excesivos. La configuración actual usa temperatura 0.3, máximo de 200 tokens generados, ocho hilos de CPU y lote de inferencia de 64.

7. API e Interfaz de Usuario

El backend se expone con FastAPI. El endpoint `POST /api/query` ejecuta la primera recuperación y devuelve documentos, consulta expandida, puntuación local máxima y un `session_id`. El endpoint `POST /api/query/feedback` recibe la selección del usuario, aplica Rocchio, recupera de nuevo y genera la respuesta final.

La interfaz React/Vite implementa tres estados: consulta inicial, selección de evidencia y respuesta final. En la primera fase el usuario redacta una pregunta en lenguaje natural y la aplicación muestra una señal de longitud cuando la consulta es excesivamente verbosa. Esta decisión responde a la limitación descrita en la Sección de recuperación probabilística: en un modelo suavizado con Dirichlet, cada término adicional puede introducir ruido si no expresa la necesidad de información.

En la segunda fase, la aplicación ya no muestra cada chunk como si fuera un documento independiente. Los resultados se agrupan por una clave canónica de documento, priorizando la URL y, en su defecto, el título. Cada tarjeta representa un artículo y muestra cuántos fragmentos fueron recuperados de ese mismo documento. Al marcar un documento como relevante, la interfaz envía internamente los identificadores de todos sus chunks asociados para que Rocchio conserve la señal completa sin saturar visualmente al usuario.

En la tercera fase, la respuesta final se acompaña de dos vistas compactas: una nube de términos ponderados por Rocchio y una grilla de documentos fuente agrupados. Esto evita presentar los resultados “uno debajo del otro” como una lista plana de chunks y hace más clara la relación entre respuesta, términos de reformulación y evidencia documental.

8. Búsqueda Web como Respaldo

Cuando la máxima relevancia local no alcanza el umbral configurado, $\tau = 0,5$, el sistema activa búsqueda web:

$$\max_{d \in D_{\text{local}}} s(d, q) < \tau \implies \text{WebSearch}(q). \quad (18)$$

El buscador usa DuckDuckGo y accesos directos a otros motores configurables. Después extrae texto de las páginas, descarta resultados sin contenido y

fragmenta los documentos con el mismo `DocumentChunker`. Los documentos web pueden incorporarse a Chroma, de modo que consultas posteriores se benefician de información que no estaba en el corpus inicial.

9. Evaluación Técnica

El diseño híbrido ofrece ventajas claras. El modelo probabilístico conserva sensibilidad a nombres de productos, cantidades, siglas y especificaciones; el modelo vectorial mejora la cobertura semántica; WRRF permite mezclar ambos sin normalizaciones frágiles; y Rocchio introduce una señal explícita del usuario antes de la generación final. La arquitectura también es reproducible: el corpus está persistido en JSONL, los índices se guardan en disco y los parámetros principales se exponen mediante variables de entorno.

Las principales limitaciones se concentran en producción, mantenibilidad y experiencia de uso. Primero, la generación local con Llama 3.1 en CPU introduce latencia perceptible, sobre todo con contextos largos. El hardware disponible obliga a trabajar con una ventana de contexto reducida, lotes pequeños e inferencia cuantizada; esto limita la longitud de evidencia que puede entregarse al modelo y vuelve más costosa la experimentación con consultas múltiples. Un despliegue con GPU, Metal o un servidor de inferencia dedicado permitiría reducir estos tiempos y probar modelos de mayor capacidad.

Segundo, las sesiones de feedback se mantienen en memoria, por lo que se pierden al reiniciar el proceso y no escalan horizontalmente sin una capa externa. Tercero, la expansión por coocurrencia puede introducir ruido en consultas con identificadores concretos, por ejemplo modelos exactos de CPU o móviles. Cuarto, las consultas demasiado verbosas siguen siendo un punto débil: aunque la interfaz ahora advierte este caso, el sistema todavía no implementa una etapa formal de reducción de consulta, extracción de entidades o ponderación diferencial de términos. Quinto, la fragmentación por ventanas solapadas puede recuperar varios chunks del mismo documento; la interfaz lo mitiga agrupándolos visualmente, pero la recuperación y el ranking aún podrían incorporar diversidad por documento antes de mostrar resultados. Por último, el ranking final ordena por metadatos de puntuación y convendría conservar de forma explícita la puntuación WRRF fusionada para auditoría completa.

10. Mejoras Futuras

La primera mejora recomendada es desacoplar el estado de sesión mediante Redis o una tabla persistente, manteniendo trazabilidad de consultas, documentos mostrados y selecciones de relevancia. La segunda es acelerar la generación con GPU, Metal o un servidor especializado como vLLM cuando el despliegue lo permita. La tercera es hacer adaptativa la expansión: consultas cortas y ambiguas deberían expandirse más, mientras que consultas con códigos de producto, cifras o nombres propios deberían expandirse poco o nada.

También se recomienda introducir una etapa explícita de reducción de consultas verbosas. Una solución práctica sería detectar stopwords residuales, entidades, números, nombres de producto y términos técnicos, y construir una consulta ponderada que conserve la intención principal sin descartar completamente el texto original. Otra mejora complementaria es aplicar diversidad por documento en el *top-k*: si varios chunks pertenecen al mismo artículo, el sistema puede mostrar primero el mejor fragmento de cada documento y dejar los demás como evidencia expandible. Esta estrategia reduciría redundancia y haría más natural el uso de Rocchio, porque el usuario evaluaría artículos completos en lugar de una lista repetitiva de fragmentos.

También sería conveniente añadir una evaluación cuantitativa con consultas etiquetadas y métricas como Precision@k, Recall@k, MRR y nDCG. Para un ranking R y una relevancia graduada rel_i , nDCG@k permitiría medir la calidad posicional:

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}, \quad nDCG@k = \frac{DCG@k}{IDCG@k}. \quad (19)$$

Estas métricas permitirían comparar LM, vectorial, WRRF y Rocchio con evidencia experimental, no solo por inspección cualitativa.

11. Conclusiones

El proyecto implementa un SRI-RAG completo y coherente para noticias tecnológicas en español. Su principal fortaleza es la combinación de técnicas: extracción propia del corpus, normalización lingüística, índice invertido, recuperación semántica, fusión por ranking, retroalimentación Rocchio y generación local controlada por contexto. La formalización matemática muestra que cada componente tiene una función clara dentro de la tubería.

El sistema todavía requiere endurecimiento para producción: persistencia de sesiones, medición formal de calidad, optimización de latencia y documentación sincronizada con la configuración real. Aun así, la base técnica es sólida para un prototipo avanzado de recuperación aumentada por generación, especialmente en escenarios donde la trazabilidad de fuentes y el control local del modelo son requisitos importantes.

Referencias

1. Ponte, J.M., Croft, W.B.: A language modeling approach to information retrieval. In: Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 275–281. ACM (1998)
2. Zhai, C., Lafferty, J.: A study of smoothing methods for language models applied to ad hoc information retrieval. *ACM Transactions on Information Systems* 22(2), 179–214 (2004)

3. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: Proceedings of SIGIR, pp. 758–759. ACM (2009)
4. Rocchio, J.J.: Relevance feedback in information retrieval. In: Salton, G. (ed.) *The SMART Retrieval System: Experiments in Automatic Document Processing*, pp. 313–323. Prentice-Hall (1971)